

Programming with Shared Memory

High Performance Computing (HPC)
Lecture 7.2

Lecture Overview



Sequential Consistency



Parallel Constructs



Dependence Analysis



OpenMP



Autoparallelization



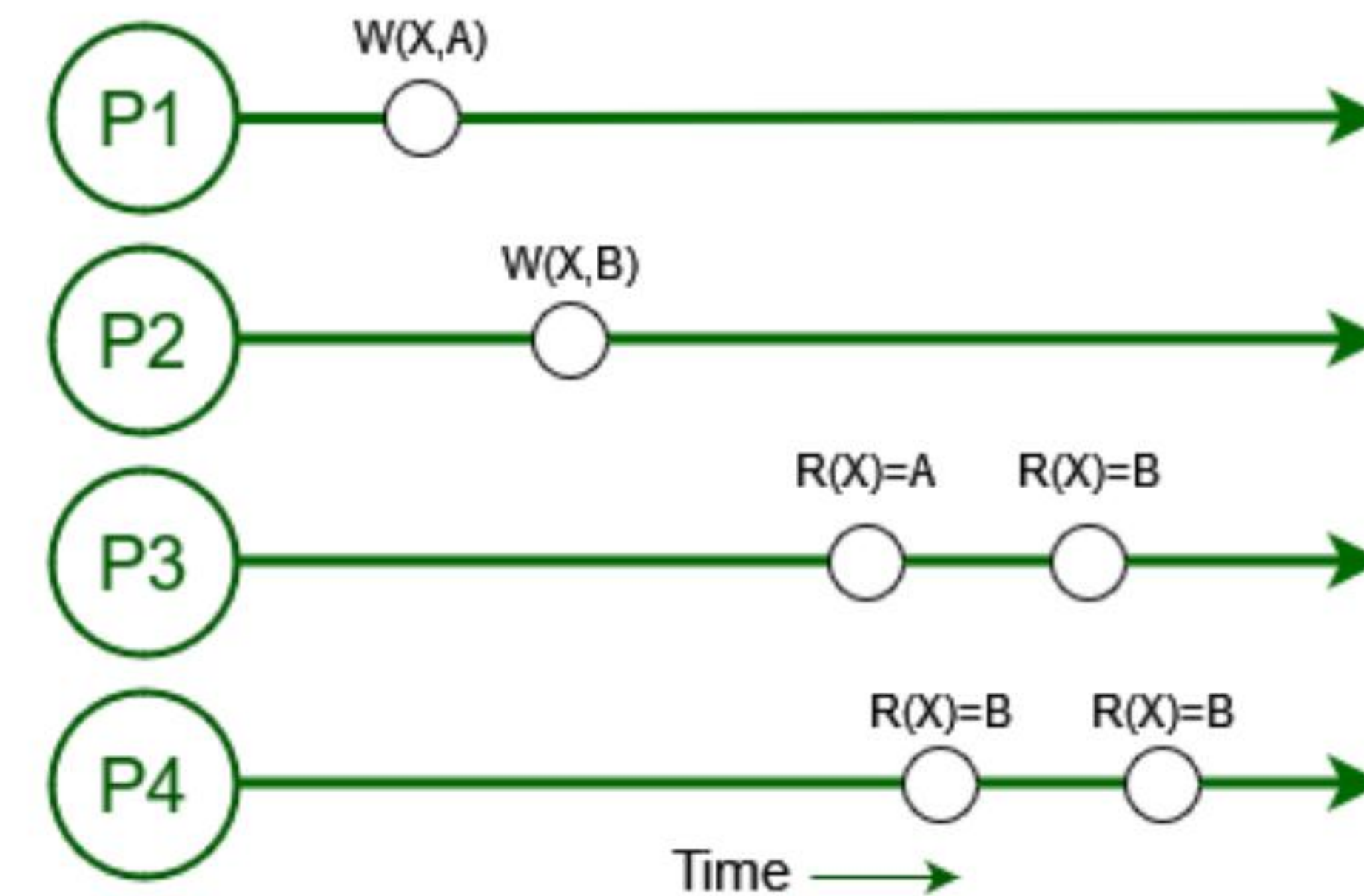
Further Reading

Sequential Consistency

A parallel program must act like a sequential one:

- > Produces the same result as the sequential version.
- > Independent of statement execution order.
- > **Correct interleavings** always lead to deterministic output.

Sequential Consistency in Distributive System



Example

Consistency Example

Scenario A

- > Same Instructions
- > Execution Order 1
- > Result: **X**

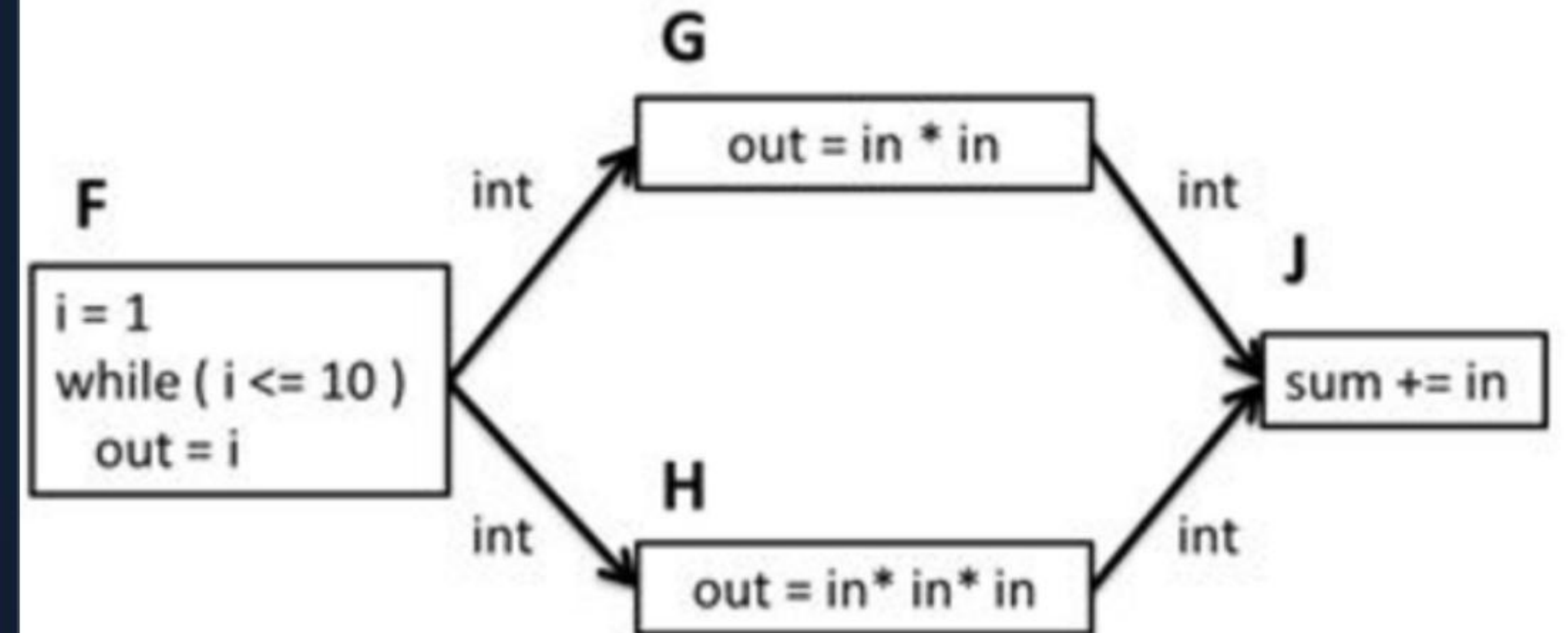
Scenario B

- > Same Instructions
- > Execution Order 2
- > Result: **X**

Essential for correctness in parallel programs.

Data Flow (Implicit Parallelism)

- > Execution is driven by data availability.
- > Operations wait until operands are ready.
- > Dependencies inherently determine the execution order.
- > Enables **implicit parallelism** without explicit thread management.



Explicit Parallel Programming

Data Declarations

- > **Shared Data:** Visible to all threads.
- > **Private Data:** Separate copy per thread.

⚠ Warning: Private pointers to shared data can be unsafe.

```
shared int x; private int y; // x is  
accessible by T1, T2... // y is  
unique to each thread
```

The 'par' Construct

```
par { S1; S2; ... Sn; }
```

Concurrent Execution

- > All statements within the block execute concurrently.
- > Each statement (S1, S2... Sn) runs in a separate thread.
- > Used for task parallelism.

The 'forall' (parfor) Construct

```
forall (i = 0; i < n; i++) { S1; S2;  
}
```

Loop Parallelism

- > Each loop iteration runs in parallel.
- > Statements *inside* a single iteration execute sequentially.
- > Ideal for data parallel tasks.

Safe Parallel Execution



Parallelism $\neq$ Syntax

It depends entirely on **data flow**.

Operations must strictly wait for their operands to be valid. Sequential consistency must be preserved at all times.

Bernstein's Conditions

Two processes P_1 and P_2 can run concurrently if:

$$\begin{array}{lcl} I_1 & \cap & O_2 = \emptyset \\ I_2 & \cap & O_1 = \emptyset \\ O_1 & \cap & O_2 = \emptyset \end{array}$$

I = Read Set (Input)

O = Write Set (Output)

Dependence Analysis



Compiler Role

Performed automatically
by the compiler.



Validation

Ensures Bernstein's
conditions are not
violated.



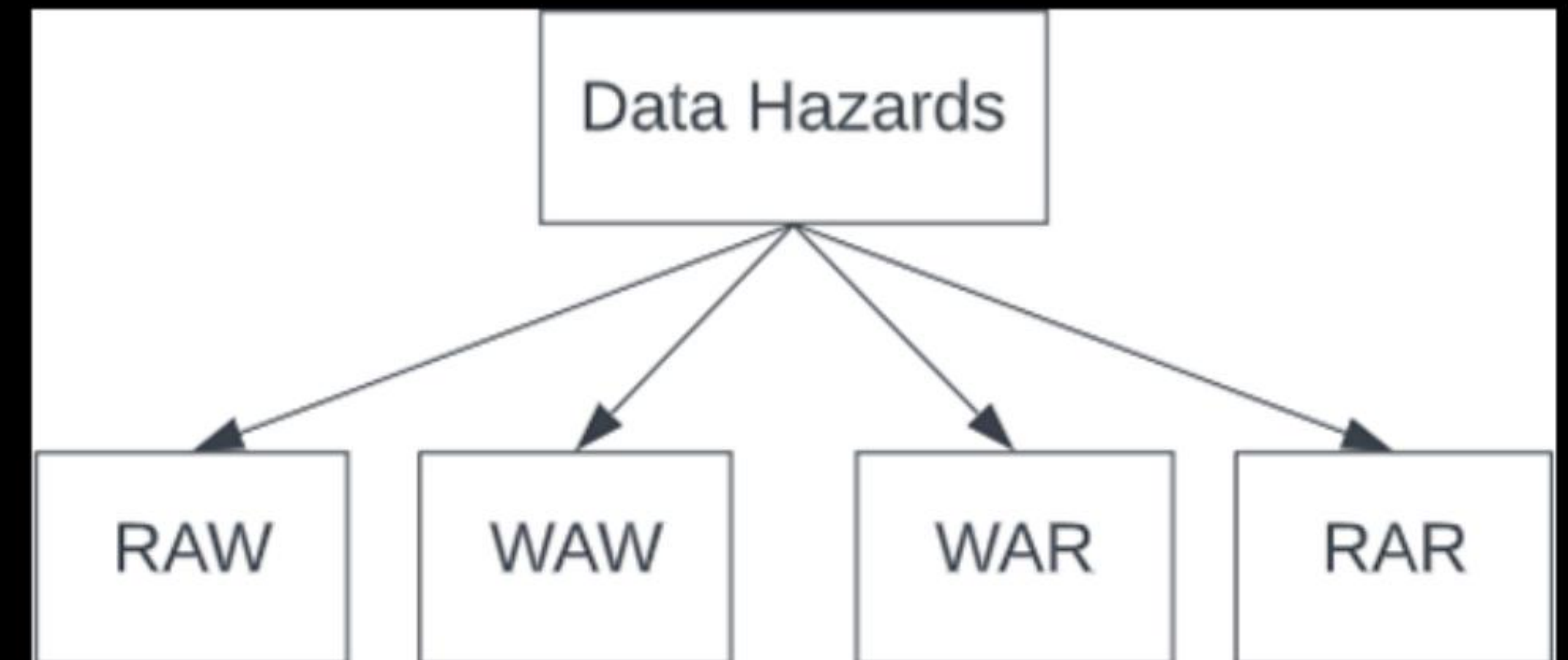
Goal

Optimization & Safe
Parallelization.

Types of Data Dependence

- > **RAW (Read After Write):** True dependence.
- > **WAR (Write After Read):** Anti-dependence.
- > **WAW (Write After Write):** Output dependence.

Any of these may prevent safe parallel execution.



Bernstein's Conditions & ILP

Instruction-Level Parallelism

Bernstein's logic underpins modern CPU pipelines.

Safe Reordering

Instructions are reordered for speed only when no data conflicts exist.

Bernstein's Conditions in Loops

- > A loop is parallelizable **only if** any execution order produces the same result.
- > Heavily depends on array indexing logic.
- > **Key:** Data Independence between iterations.
- > Example: Independent array accesses (e.g., $A[i] = B[i] + C[i]$).

End of Lecture 7.2

Consistency

Sequential consistency is mandatory.

Dependencies

Data dependencies control parallelism.

Safety

Bernstein's conditions ensure safety.

Analysis

Compiler analysis is essential.

Embarrassingly Parallel Algorithms

High Performance Computing (HPC)
Lecture 8

Topics Covered

Ideal Parallelism

Master–Worker

Processor Farms

Mandelbrot Set

Monte Carlo

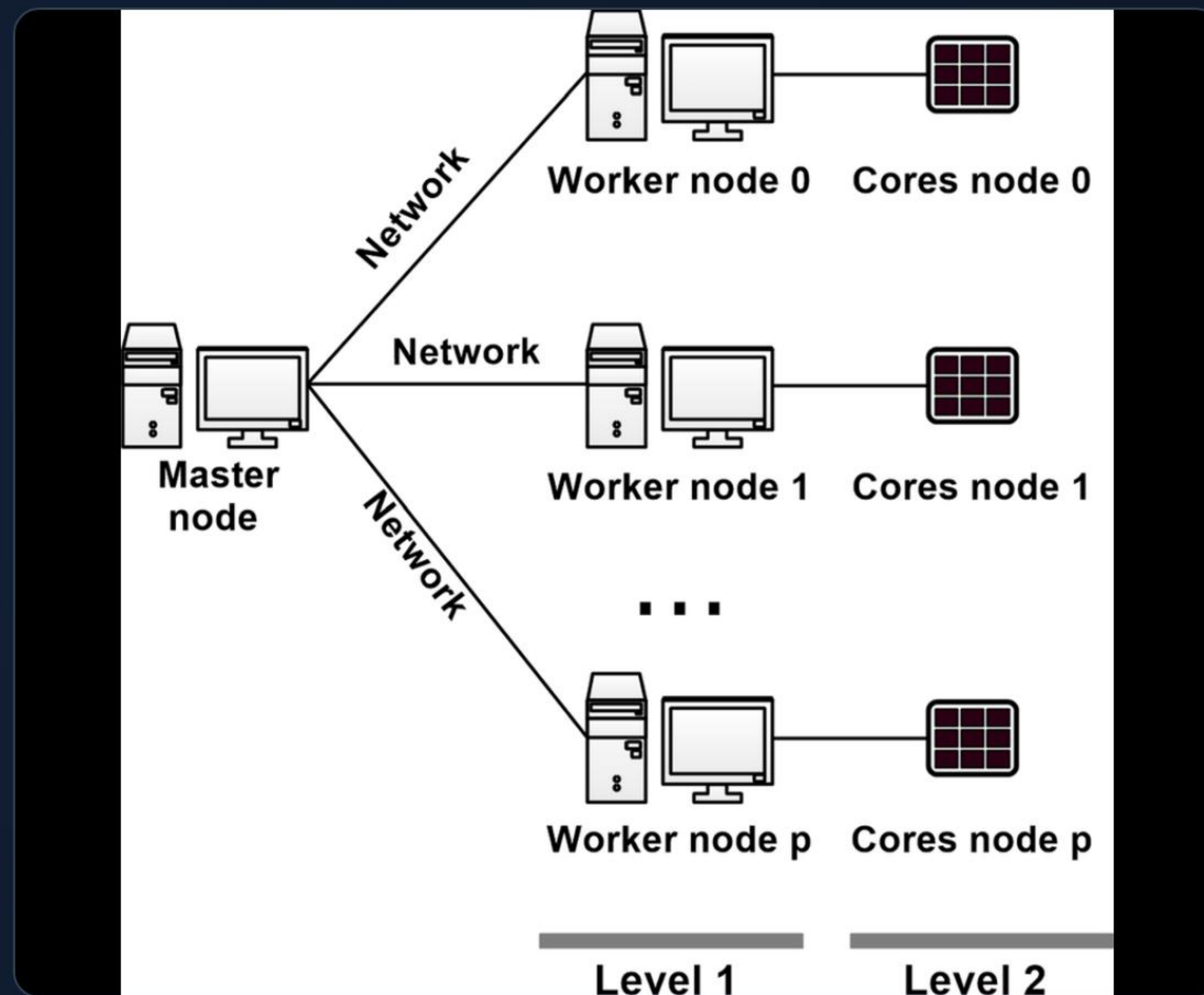
Load Balancing

Ideal (Embarrassingly) Parallel

- > Computation divides into **fully independent tasks**.
- > Zero communication required between tasks.
- > Also known as "Naturally Parallel".
- > **Result:** Maximum possible speedup.

Master-Worker Paradigm

- > No communication between workers.
- > **Master:** Sends initial data & collects results.
- > **Worker:** Processes data independently.
- > Communication happens only at start and end.



Parallel Tradeoffs

Parallel Time

$$t_p = t_{\text{comm}} + t_{\text{comp}}$$

Speedup

$$S_p = \frac{t_s}{t_p}$$

Best performance when $t_{\text{comp}} \gg t_{\text{comm}}$

Communication Cost

- > Communication can be more expensive than computation.
- > Small tasks suffer from high overhead (latency dominates).
- > **Solution:** Overlap communication with computation to hide latency.

Ex 1: Geometrical Transformations

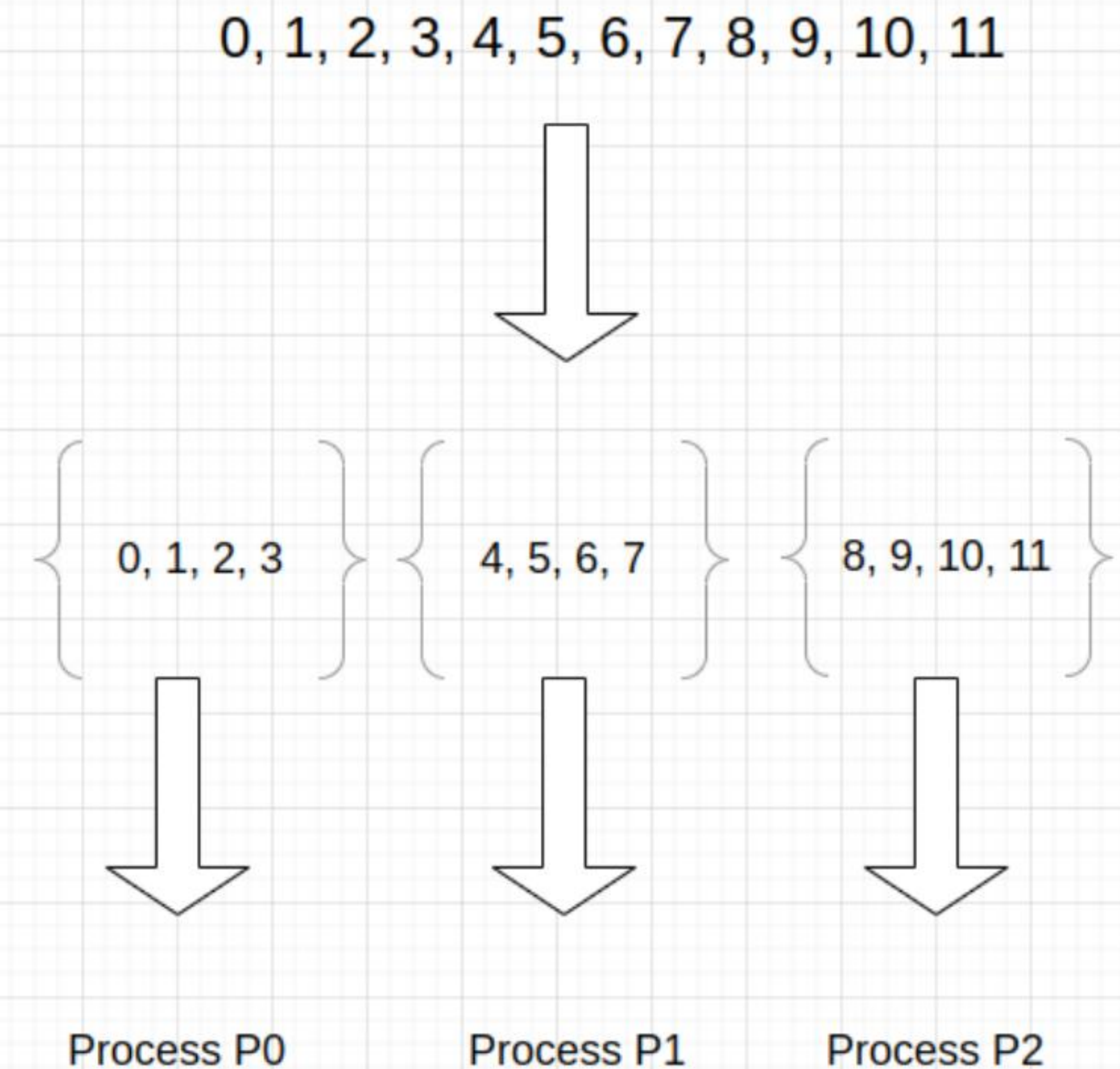
Partitioning Strategies

- > Row-based partitioning.
- > Block-based partitioning.

Operations

- > Shift, Scale, Rotation, Clip.

Block Partitioning



Naïve Master–Worker (Image)

Master Role

Distributes image rows one by one.
Collects individual results.

Worker Role

Computes new pixel position. Sends each
pixel result back.

⚠ CRITICAL: High communication overhead destroys performance.

Performance Analysis

- > Pixmap size: n^2 points
- > Sequential Time: $O(n^2)$
- > Communication: $O(P + n^2)$
- > Computation (per worker): $O(n^2 / P)$

Result: Poor computation-to-communication ratio.

Ex 2: Mandelbrot Set

Each pixel is computed independently based on the recurrence relation:

$$z_{k+1} = z_k^2 + c$$

The "Embarrassingly Parallel" ideal.

Mandelbrot Color Computation

- > Iterate until $|z| > 2$ or maximum iterations reached.
- > Pixel color depends solely on the iteration count.
- > **No shared data** between pixels.
- > Perfect for parallel rendering.

Simple Master–Worker (Mandelbrot)



Worker

Computes pixel color. Sends (x, y, color) tuple.

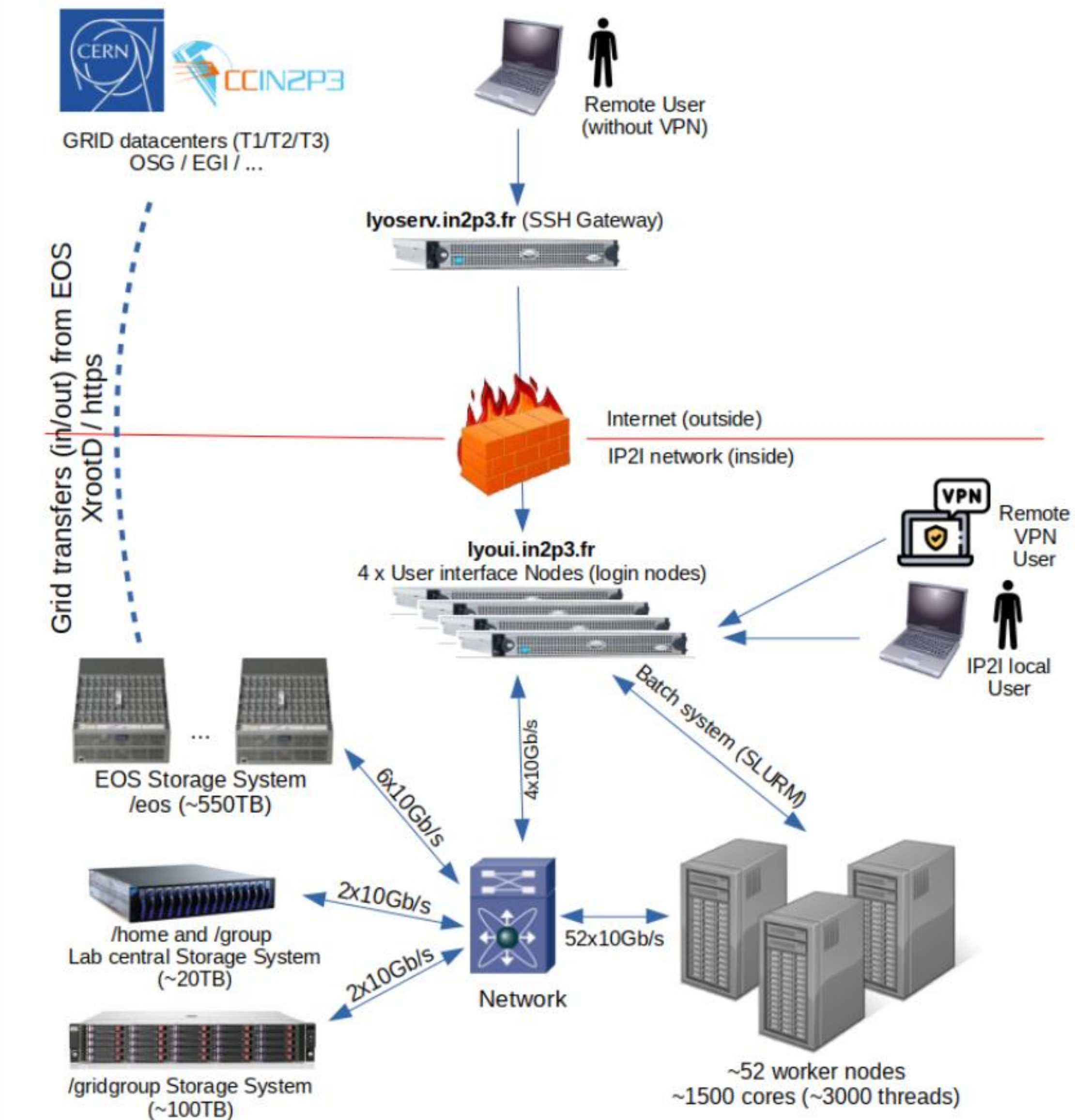


Master

Collects and displays pixel results on screen.

Processor Farms

- > Utilizes a **Shared Work Pool**.
- > Workers fetch tasks, compute, and return results.
- > Provides **Automatic Load Balancing**.



Load Balancing



The Goal

Distribute work evenly across all processors to minimize idle time.



Causes of Imbalance

1. Tasks with different execution times.
2. Processors with different speeds.

Static Load Balancing

- > Tasks are assigned **before** execution begins.
- > **Techniques:** Round Robin, Randomized, Recursive Bisection.
- > **Drawback:** Difficult to estimate task cost accurately in advance.

Centralized Dynamic Load Balancing

Central Queue

A single Master manages a global task queue.

Dynamic Fetch

Workers request new tasks whenever they become idle.

Decentralized Dynamic Load Balancing

- > No single master bottleneck.
- > Uses "Mini-masters" or distributed queues.
- > Significantly reduces contention for the task pool.
- > Scales better for large numbers of processors.

Fully Distributed Work Pool

Pull Model

Idle workers request tasks from busy neighbors.

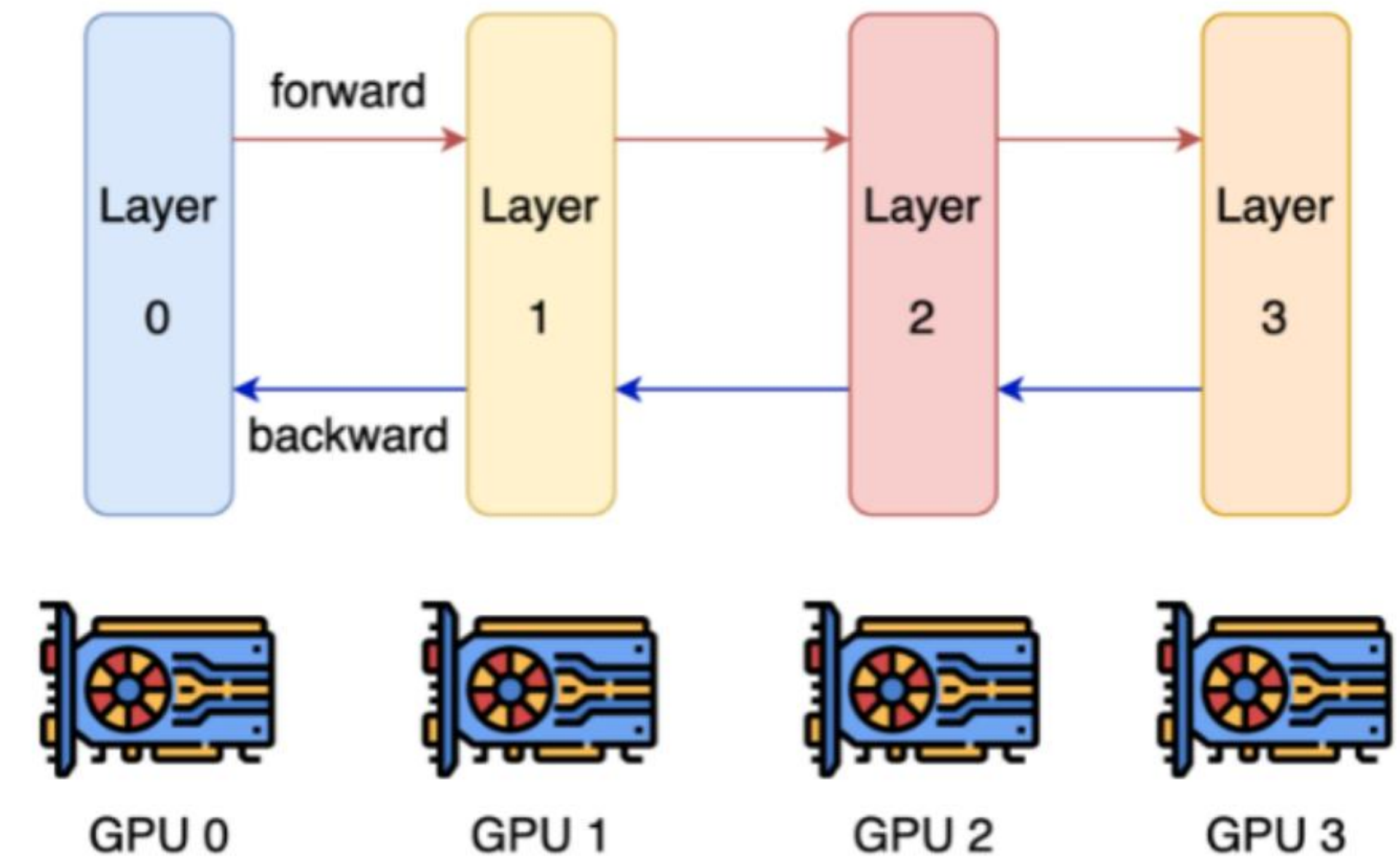
Push Model

Busy workers offload tasks to others.

Worker Selection: Nearest Neighbor, Round Robin, or Random.

Worker Pipeline

- > Workers organized in a linear line or ring.
- > Tasks flow sequentially between workers.
- > Reduces idle time by keeping the pipe full.



Final Conclusion



Embarrassingly parallel algorithms provide **maximum speedup**.



Communication cost is the main challenge to scalability.

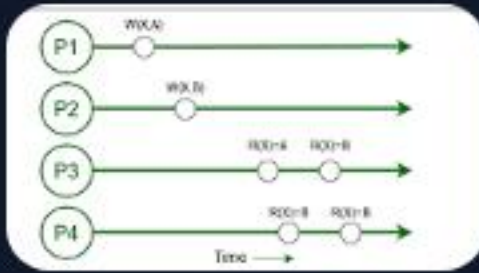


Load balancing is critical for efficiency.



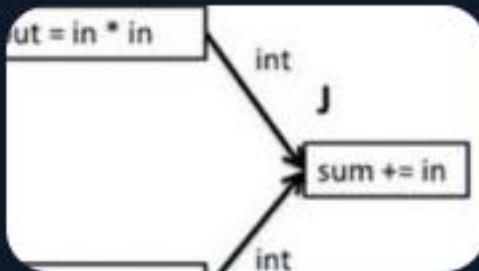
Dependence analysis ensures safe execution.

Image Sources



<https://media.geeksforgeeks.org/wp-content/uploads/20230310184135/Sequential.png>

Source: www.geeksforgeeks.org



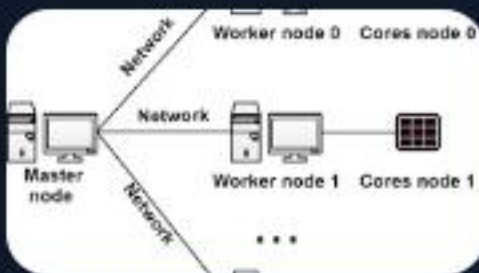
<https://www.intel.com/content/dam/docs/us/en/developer-guide-api-reference/2021-6/5CDC5EF2-A391-4C23-B9D0-AF1DD761C619-low.png>

Source: www.intel.com



https://miro.medium.com/v2/resize:fit:1164/1*-mLukbjbeS20o5OpA-_j2Q.png

Source: [medium.com](#)



https://miro.medium.com/1*fx1xvD2LF7bwc5qSaRn4yw.png

Source: medium.com



https://miro.medium.com/0*z7d5OmiUroFwyRpg

Source: medium.com



https://upload.wikimedia.org/wikipedia/commons/thumb/a/a3/Mandelbrot_set_10000px.png/2560px-Mandelbrot_set_10000px.png

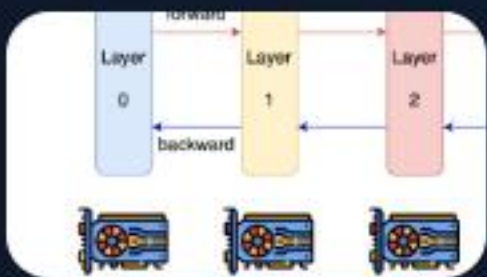
Source: commons.wikimedia.org

Image Sources



<https://ip2i.pages.in2p3.fr/calcul/documentation/calcul/slurm-architecture.png>

Source: ip2i.pages.in2p3.fr



https://miro.medium.com/1*5an_Pcz6uOVivXsQ52jz7Q.png

Source: [medium.com](https://miro.medium.com)